

Animation Composer System (ACS) - Documentation

Welcome

Welcome to the Animation Composer System (ACS) Bundle Documentation. Here you will find tutorials, as well as key information about the classes and methods in the package. I hope you will be able to find what you are looking for.

If there's anything you're missing here, feel free to ask me anything in the [Discord](#) server.

Why ACS? 💡

The Anim Composer asset is inspired by Unreal Engine's Anim Montages. It removes the requirement to use Unity's Animator and provides a powerful way to organize and play single animation clips with a layer of custom game logic.

While a standard Unity animation clip only defines how bones move, an Anim Composer takes care of blending between animations, and provides other key options like setting the anim to loop, changing the rate or previewing attached items. It also allows you to add timed behaviors (`ActionBlocks`) right onto the timeline.

This is incredibly useful for a variety of use cases, such as:

⚔️ **Weapon Swings:** Play a "whoosh" sound effect or enable sword damage collider.

🏃 **Motion Warping:** Moving the character by a set amount during a dodge or attack.

💣 **Vfx and Sfx:** Playing specific visual effects or sounds for a special ability.

👣 **Footsteps:** Detect ground below and play corresponding footstep SFX and VFX.

Jump right in



Quickstart

Learn all you need to start working with AnimComposers.



Styling the AnimComposer Editor

Discover how to customize the AnimComposer editor to match your workflow and style preferences.



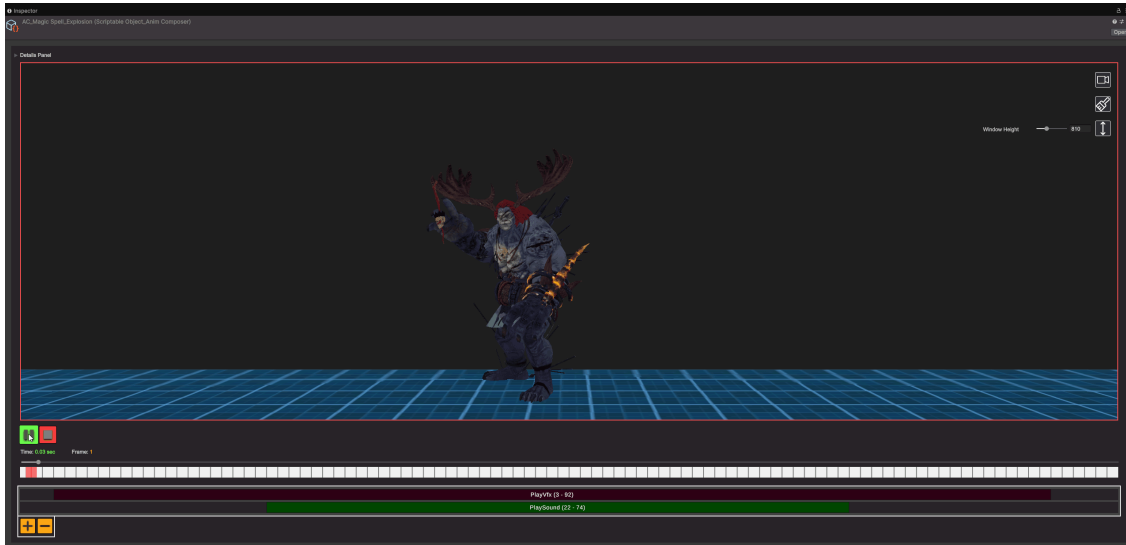
Sample Material

Explore the sample scene and objects included in this bundle, and instantly try fully functional examples.

Outline

Quickstart

In this section, we will create a character or use a preexisting character in our projects and attach the `AnimCoordinatorComponent`. We will then create and edit our first `AnimComposer` object, and show how you can play it at runtime. Finally, we will show how to create and implement a custom `ActionBlock` element, and how it can be used inside `AnimComposers`.



Anim Composer Editor



Setting up your character.



Setting up the AnimCoordinator component.



AnimComposers Overview: Manage, Preview, and Control Animations



Executing AnimComposers at runtime.



ActionBlocks Overview: Attach precise and flexible logic to animations



Assigning ActionBlocks to AnimComposer.



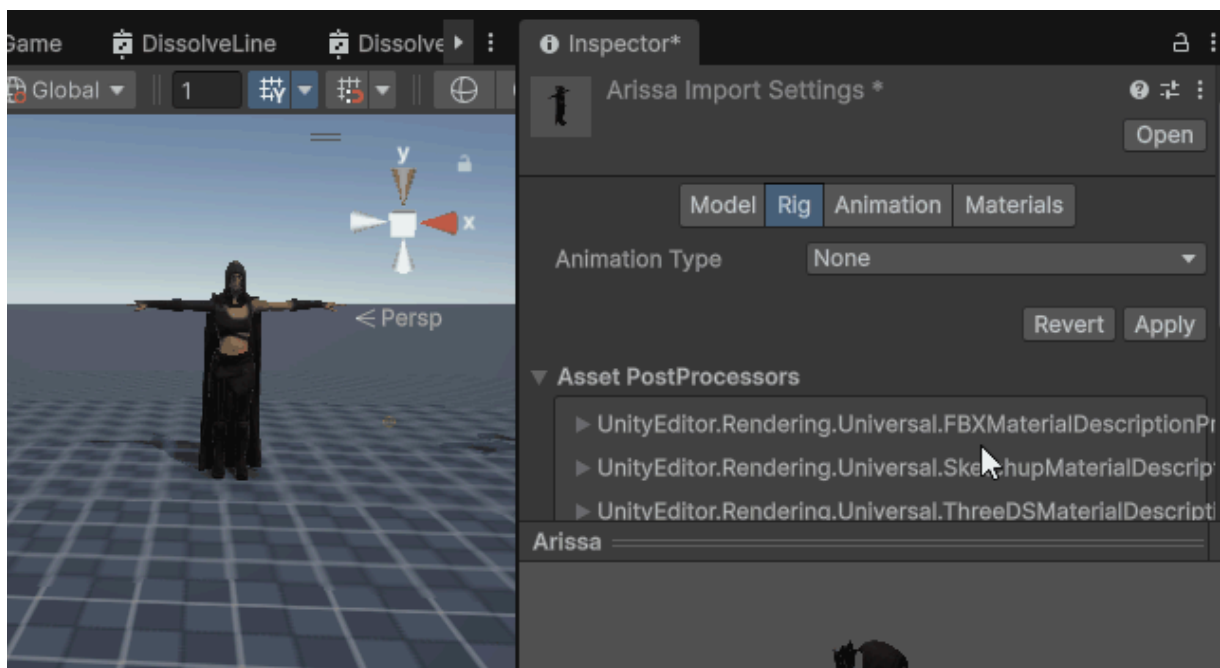
Setting up your character.

To create your character in Unity, start by importing a 3D model:

- **Drag and drop** your `.fbx` model into the scene.
 - You can use a character model exported from Mixamo or any other `.fbx` file.
- **Alternatively**, you can import a standard character directly through the **Unity Package Manager**.

For more details, refer to the Unity [documentation](#).

1. Create a **Humanoid** Avatar from this character.



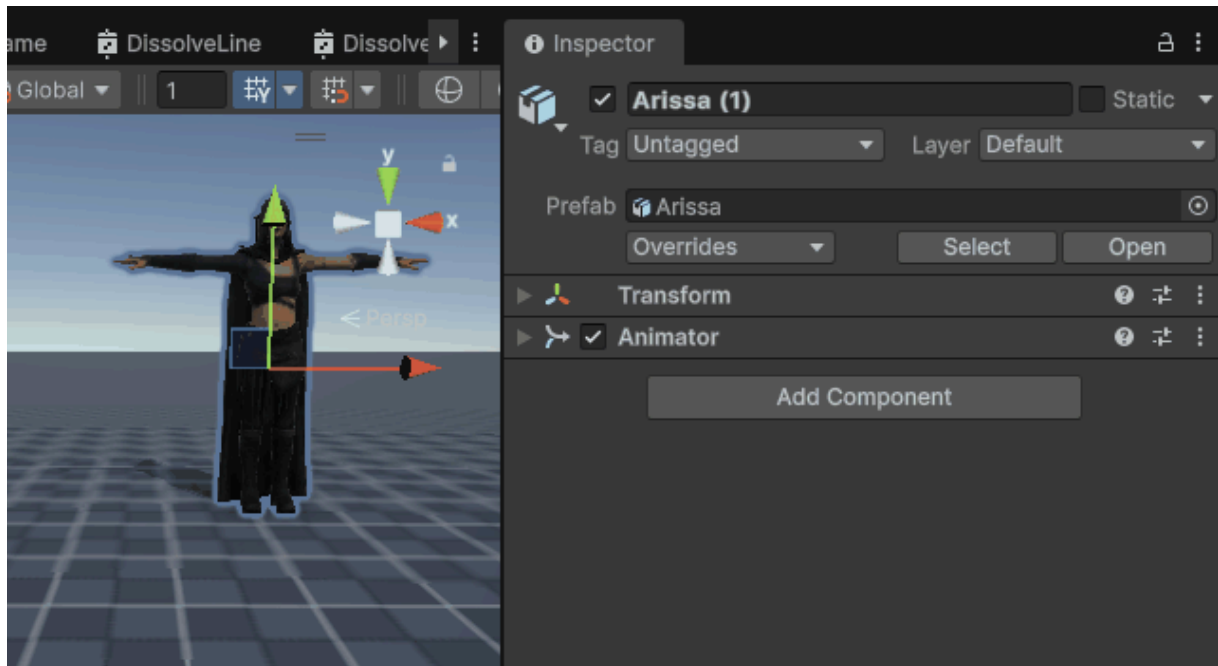
Setting Animation Type to Humanoid and creating Humanoid Avatar from Mesh

2. **Create or extend a pre-existing MonoBehaviour** (e.g., `AnimDispatcherComponent`). Add the following code:

```
void OnAnimatorMove(){}}
```

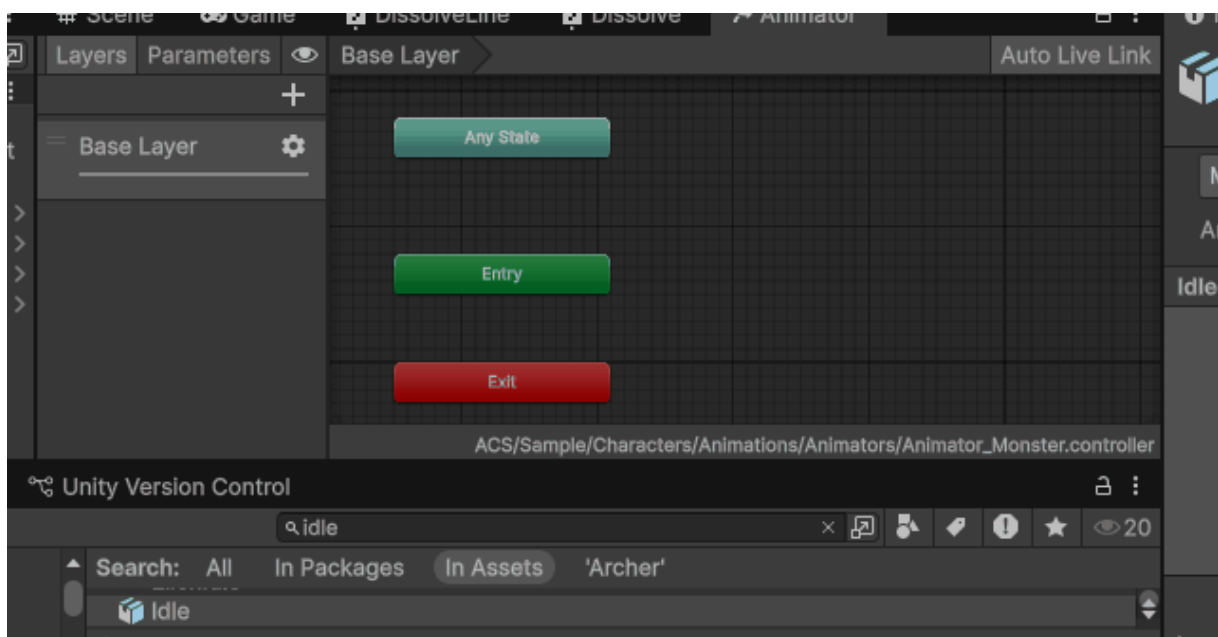
This allows us to control whether animations use root motion

3. Drag the `.fbx` character into the scene, then add the newly created `AnimDispatcherComponent`.



Assigning AnimDispatcher

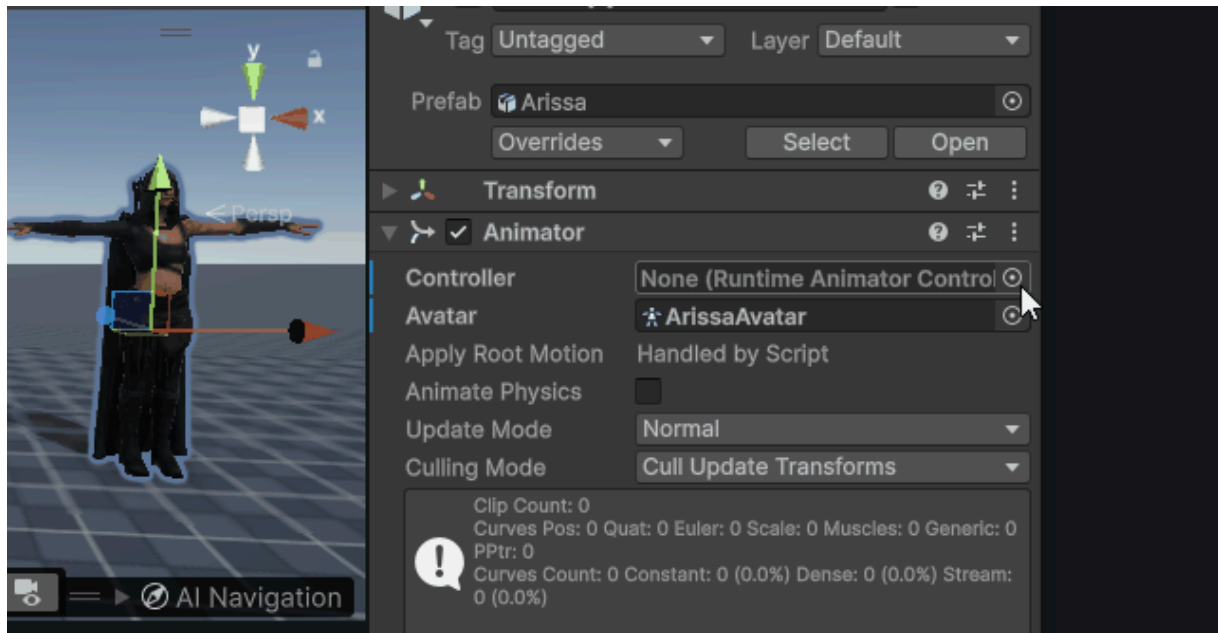
4. **Create a new `AnimatorController`** (Right-click → Create → Animation → `Animator Controller`). Then, drag and drop your character's **Idle** (base animation) into it.



Assigning Idle State to AnimatorController

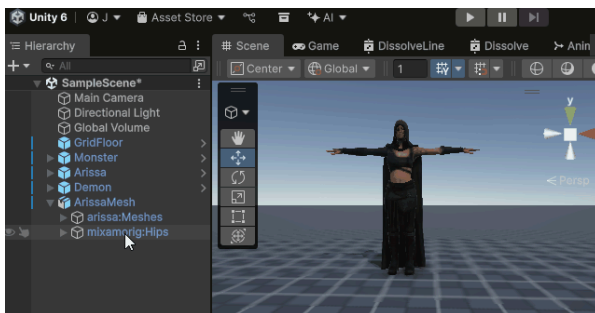
- Alternatively, you can define a layer in the `AnimCoordinator` for Idle animations, and play different Idles using ACS. More on that in [Configuring the AnimCoordinatorComponent](#).

5. Assign the newly created `AnimatorController` to the Animator component of your character in the scene.

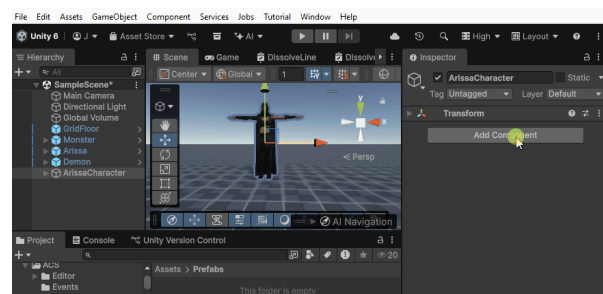


Assigning the AnimatorController to the Animator

6. Finally, drag your character (with the added components) back into the Assets tab to create a Prefab—and your character is ready! Before this step, you may want to create a parent GameObject to hold all other MonoBehaviours and attach a `CharacterController`, `Rigidbody`, and/or `CapsuleCollider` as needed.



Creating Parent GameObject



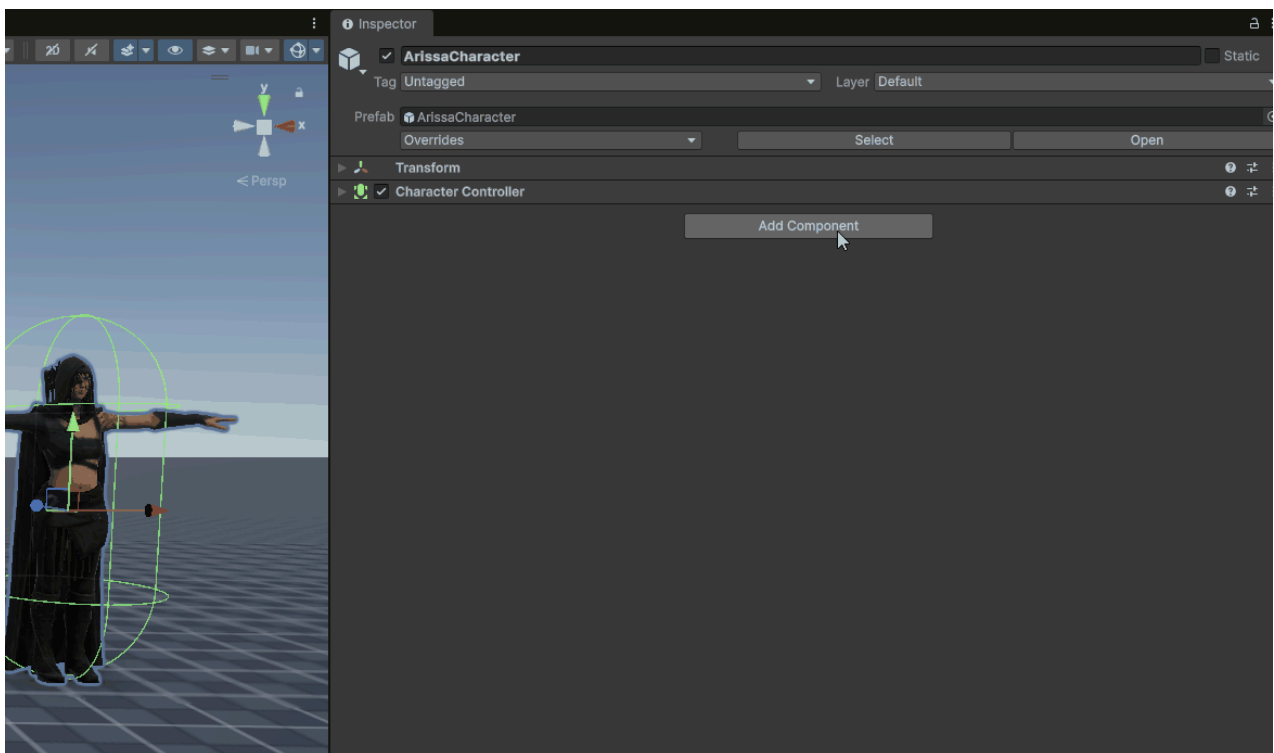
Adding CharacterController and saving as Prefab

Setting up the AnimCoordinator component.

The **AnimCoordinator** is the main component that handles playing and stopping animations on your character.

- It lets you define multiple **animation layers**, each with its own **Avatar Mask**.
- This makes it possible to **combine animations additively** (e.g. running + crouching) or **override specific bones** (e.g. playing a custom upper-body animation while the legs keep running).

To use it, simply add the **AnimCoordinator** component to the **Root GameObject** of your Character Prefab.



Animation Layers

The **Animation Layers** list lets you define extra layers on top of your character's base Animator. Each entry in this list includes:

- An **Avatar Mask** (which controls which bones the layer affects).
 - An **IsAdditive** property (to choose between *additive* or *override* blending).
 - A **Layer Weight** value (between **0** and **1**) that controls how strongly the layer influences the final pose.
-

How layers stack

- The **base Animator Layer** (not shown in this list) is always considered **Layer 0**. This is where you usually place your Idle animation.
 - The first element you add to this list becomes **Layer 1**, the next is **Layer 2**, and so on.
 - Layers are applied in order: higher layers play **on top of** lower layers.
 - Example: Layer 2 blends or overrides Layer 1 → Layer 1 blends or overrides Layer 0 → Layer 0 blends or overrides the base Animator Layer.
-

Typical setup

- Start by creating **one layer without an Avatar Mask**. Any animation in this layer will affect the entire skeleton.
 - To control only specific bones, add more layers with custom **Avatar Masks**. You can then decide whether they are **Additive** (blend on top of previous motion) or **Override** (replace the motion on those bones).
-

Layer Weight

The **Layer Weight** setting is included in each layer entry.

- Set it between **0 (no influence)** and **1 (full influence)**.
- Most of the time you'll want to keep this at **1**.
- For some additive animations (e.g. crouch pose in the provided template), setting the weight to a lower value can help you achieve a more natural blend with the base animations.

 If you want more control over different **Idle animations**, it can be helpful to create an additional **layer at index 0** in the Animation Layers list with **no Avatar Mask**.

- This allows your **looping Idle animation** to play continuously.
- You can then switch idle animations **contextually**, for example:
 - When the character equips an item
 - When the character gets hurt

You can also add **another layer right after this one** (also with no Avatar Mask) for **non-looping or state-based animations**, giving you more flexibility without affecting the base Idle.

If you choose not to add these extra layers, the system will still work: once the AnimCoordinator finishes playing any other animations, it will **automatically return to the Idle state** you defined in the base AnimatorController.

 Every variable in the Inspector comes with a **descriptive tooltip**.

- Simply **hover your mouse** over a field to see helpful information about what it does.
- This makes it easier to understand each setting without needing to check the documentation.

AnimComposers Overview: Manage, Preview, and Control Animations

If you've used **Animation Montages in Unreal Engine**, AnimComposers will feel familiar.

- **AnimComposers** are wrappers for animations that let you:
 - Configure key details of the animation
 - Bind functionality to specific frames using **ActionBlocks** (similar to Unreal's Anim Notifies and Anim Notify States)
-

Key Benefits

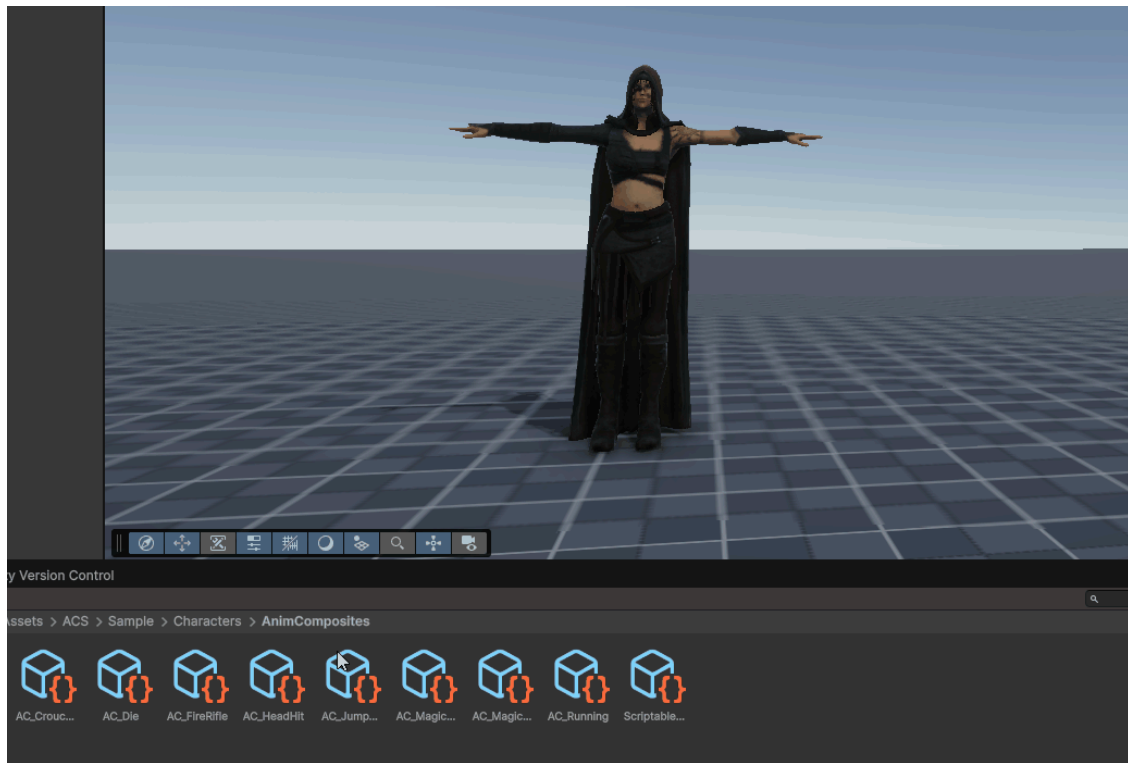
- You can create **different AnimComposers for the same animation** if you want slightly different behaviors.
 - You can **reuse the same AnimComposer across multiple characters** to minimize asset duplication and reduce project size.
 - For **Humanoid characters**, animations are automatically retargeted, so you don't need to worry about mismatched rigs.
 - AnimComposers make it easier to **manage animations without cluttering your AnimatorController** with states, transitions, and parameters.
 - Similar to [Animancer](#) ↗, AnimComposers allow you to **link animation directly to gameplay logic**, but in addition, offer **much more advanced previewing** than Unity's default Animation Preview Window, including:
 - **VFX**
 - **Sounds**
 - **Environment**
 - **Preview Objects**
 - **Hitboxes**
 - **Materials**
-

Now that you understand how AnimComposers can help streamline your workflow, let's see **how to create and edit them in your project!**

Creating and Editing a new AnimComposer.

In the **Assets** tab, **right-click** and select:

Create → ScriptableObjects → AnimComposer



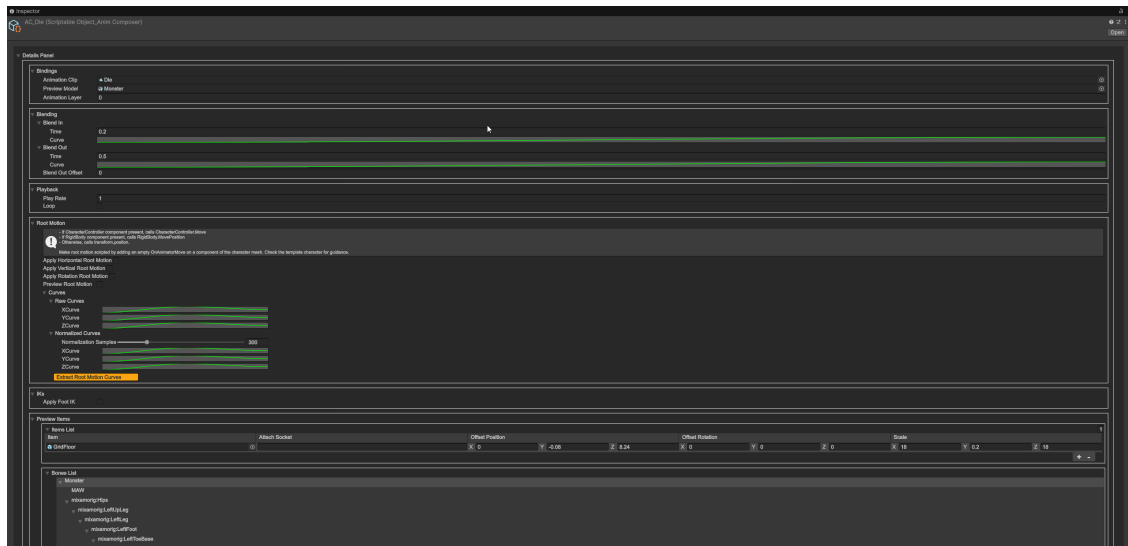
You can view and edit **AnimComposers** directly from the **Inspector** in its collapsed form.

However, for a more comfortable and detailed editing experience, it's highly recommended to open the **AnimComposer** in a full-screen window.

To do this, you have two options:

- **Double-click** the **Inspector** tab to expand it to full screen, or
- **Right-click** your **AnimComposer** asset and select **Properties**.

Details Panel



The Details Panel is where you can configure every aspect of the animation and how it behaves during playback. It's organized into several expandable **foldouts**, each controlling a specific set of parameters.

• Bindings

This contains the animation and preview model settings.

- **Animation Clip** — The animation clip that this composer will play.
- **Preview Model** — The model used to preview the animation in the lower preview window.
- **Animation Layer** — Specifies the animation layer (as defined in the **AnimCoordinator** component) that this animation will use. This allows assigning different layers for **additive** or **override** animations.



AnimComposers work with any humanoid model — they are **not limited** by the chosen **Preview Model**.

The Preview Model is only used for visualization within the editor and does not affect which models the animation can run on at runtime.

- **Blending**

The **Blending** section allows you to control how smoothly animations transition in and out.

- **Blend In / Out Duration** — Adjusts how long the animation takes to fade in or fade out.
- **Blend Curves** — Defines the easing curves used during blending to create more natural transitions.
- **Blend Out Offset** — Determines *when* the animation begins blending out relative to its total duration.

In practical terms:

$$\text{Blend Out Start Time} = \text{Animation Length} - \text{Blend Out Time} + \text{Blend Out Offset}$$

This means the animation will begin blending out before it fully ends, based on the offset you set.

- **Playback**

The **Playback** section controls how the animation is played.

- **Playback Rate** — Adjusts the speed at which the animation runs.
The preview window below updates in real time to reflect the configured playback rate.
 - **Loop** — Determines whether the animation should **repeat continuously** or **stop once it reaches the end**.
-

- **Root Motion**

Root Motion is managed internally by the **AnimCoordinator** component. It supports both **non-physical** and **physical** character movement:

- If the character has a **CharacterController** component, motion is applied using `CharacterController.Move()`.
- If the character has a **Rigidbody** component, motion is applied using `Rigidbody.MovePosition()`.

For practical examples, check the **template characters** included with the tool.

You can configure which types of root motion to apply:

- Horizontal(X/Z translation)
- Vertical (Y translation)
- Rotation

Previewing Root Motion

Enable **Preview Root Motion** to visualize root motion directly in the animation preview window.

This setting affects the **editor preview only** — runtime animations are **not** influenced by this toggle.

Extracting Root Motion Curves

You can extract **root motion curves** from an animation clip.

This provides both **raw** and **normalized** position curves (X, Y, Z), representing how the root moves over time.

These curves can be copied and used in **custom logic** or **ActionBlocks** — for example, to create **realistic and smooth Motion Warping**.

⚠ To ensure root motion works correctly, make sure the animation's motion **is not baked into the pose** in the assigned **AnimationClip**. (See the [Animation tab](#) ↗ for more details.)

⚠ For root motion to work, you must have followed step 2 in the [Character setup](#)

 **Motion Warping** is a technique used to dynamically adjust a character's position and rotation during an animation — while preserving the natural, detailed motion of the original clip.

This allows characters to **interact seamlessly with variable environments**, such as:

- Vaulting over ledges of different **heights** or **lengths**
- Reaching or striking **enemies** positioned at varying distances

Compared to traditional linear motion adjustments, **Motion Warping** produces **smoother, more natural, and highly adaptive movements**.

It's also **simple to set up** using **ActionBlocks**.

For more info on this, please ask in the [Discord](#)  page.

- **IKs (Inverse Kinematics)**

The **IKs** section lets you enable or disable **Foot IK** for the current animation.

- **Apply Foot IK** — When enabled, this animation will trigger **OnAnimatorIK** callbacks, allowing you to integrate your own **custom** or **plugin-based IK** logic.

Additionally, the following **event channels** are raised automatically:

- **OnIKFootBlocked** — Triggered when an animation with **ApplyFootIK** enabled **starts**.
 - **OnIKFootUnblocked** — Triggered when that animation **ends**.
-

- **Preview Items**

The Preview Items foldout displays a configurable table called the **Items List**, where you can customize and visualize items directly in the preview window.

Items List

In this table, you can specify:

- **Item Prefab** — The item asset to spawn in the preview window (e.g., weapon, shield, armor piece).
- **Attach Socket** — The **character bone** where the item will be attached.
 - Leaving this field **empty** or providing an **invalid name** means the item will not attach to any bone.
- **Offset** — The **position**, **rotation**, and **scale** adjustments applied to the attached item.

Bones List

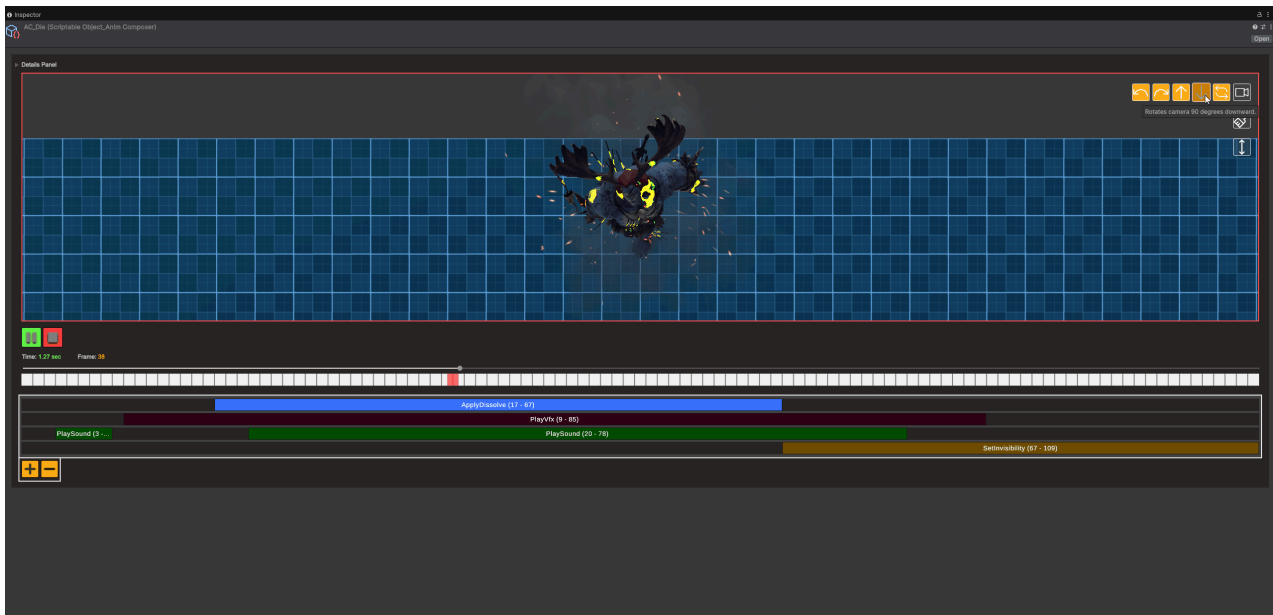
The **Bones List** displays the full hierarchy of the character's bones.

You can use it to **look up bone names** and **copy them** into the **Attach Socket** field for precise attachment.

⚠ The **Preview Items** system is **for visualization only** — it does **not appear at runtime**. It's especially useful for:

- Previewing **attack animations** with specific weapons or armor
- Testing **mobility animations** with nearby obstacles
- Ensuring **item alignment** and **socket accuracy** before play mode

Animation Preview Window



The **Animation Preview Window** allows you to preview your animation **before running the game**.

You can also preview **attached items, VFX, sounds, materials**, and other components in real time.

Camera Controls

- **Rotate** — Hold the **right mouse button** and move the mouse to rotate the camera around the preview model.
- **Zoom** — Scroll the **mouse wheel** to zoom in and out.
- **Pan** — Hold the **middle mouse button** (wheel) and move the mouse to pan the camera vertically and horizontally.

Preview Window Menus

In the top-right corner of the preview window, there are **three expandable menus**:

1. Camera Orientation Options

- Buttons to rotate the camera in **90° increments** along each axis.
- The **last button** resets the camera orientation to its **original state**.

2. Background Color Option

- Pick a **custom background color** for the preview window.

3. Screen Height Options

- Drag the **slider** to adjust the **height of the preview window** according to your preference.



The **Screen Height** and **Camera Orientation** settings will **reset** to their default values every time you select a new **AnimComposer**.

Only the **Background Color** setting will **persist** between selections.

Executing AnimComposers at runtime.

AnimComposers can be easily **played** or **interrupted** at runtime.

You can refer to the **sample scene** to see how they are **bound to player inputs** through the **AnimationTester** component.

Integrating AnimComposers Into Your Game

The sample scene provides a **minimal example** of usage.

In practice, how you bind AnimComposers to your gameplay logic will depend on your project's **architecture**.

For best results, this system shines when used together with a **well-structured, ScriptableObject-based state or ability system** — similar in concept to **Unreal Engine's [Gameplay Ability System](#)** ↗.

This approach allows you to easily define:

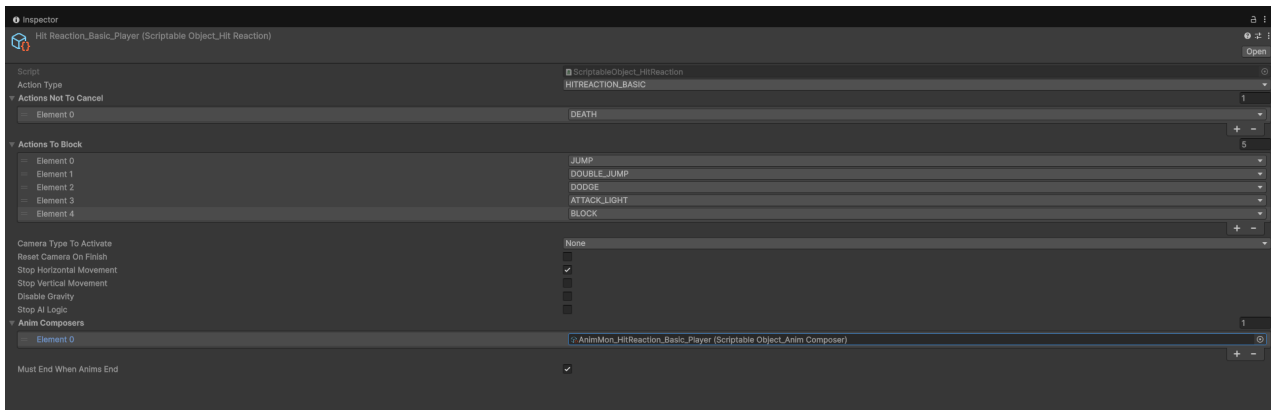
- New **abilities**
 - Their **effects**
 - Their corresponding **AnimComposers**
-

Recommended Design Approach

While the example shown is simple, I recommend introducing an **additional abstraction layer** to:

- Process **inputs**
- Handle **setup logic**
- Manage **transitions** between different AnimComposers

This separation of concerns helps keep your animation logic clean and scalable.



Custom Action (HitReaction) which extends AnimationAction class and ends when assigned AnimComposers end.

ActionBlocks Integration

That said, **ActionBlocks** can also contain all necessary logic internally — including:

- **Blocking** or **buffering** inputs during specific frames
- Preventing one **AnimComposer** from interrupting another prematurely

This makes it possible to build complex and responsive animation behaviors without relying on external state machines.

- ✓ This represents a **new paradigm** for handling animations in Unity. While it may feel unfamiliar at first, you'll likely find it **cleaner, more modular, and easier to maintain** than traditional **Animator Controller** workflows.

Public Events

There AnimCoordinator component has events to which you can subscribe from other scripts in order to link specific behaviour to them (e.g. ending the action when the animation ends). The main events are

```
/// <summary>
/// Event that is raised when an animation composer begins playing.
/// </summary>
public event AnimCoordinatorEvent_AnimStart OnAnimStart;
```

```
/// <summary>
    /// Event that is raised when an animation composer finishes or is
    stopped.
    /// </summary>
    public event AnimCoordinatorEvent_AnimEnd OnAnimEnd;
```

```
/// <summary>
    /// Event that is raised when foot ik is blocked.
    /// </summary>
    [Tooltip("Event that is raised when foot ik is blocked.")]
    [SerializeField, HideInInspector]
    protected IKFootUnBlockedChannel OnIKFootBlocked = null;
```

```
/// <summary>
    /// Event that is raised when foot ik is unblocked.
    /// </summary>
    [Tooltip("Event that is raised when foot ik is unblocked.")]
    [SerializeField, HideInInspector]
    protected IKFootUnblockedChannel OnIKFootUnblocked = null;
```

Public Functions

The **AnimCoordinator** component provides all the functions required to **manage AnimComposers** at runtime.

The most relevant functions include:

```

/// <summary>
/// Plays a single animation composer on its designated layer.
/// Interrupts any other composers currently playing on the same
layer.
/// </summary>
/// <param name="newAnimComposer">
/// The <see cref="ScriptableObject_AnimComposer"/> to play.
/// </param>
/// <param name="customBlendInTime">
/// Optional custom blend-in duration. If not provided, the
composer's
/// <c>BlendInTime</c> value will be used.
/// </param>
/// <param name="customBlendInCurve">
/// Optional custom blend-in curve to control the interpolation. If
not provided,
/// the composer's default <c>BlendInCurve</c> will be used.
/// </param>
public void PlayAnimComposer(ScriptableObject_AnimComposer
newAnimComposer,
                                float ? customBlendInTime = null,
AnimationCurve
                                customBlendInCurve = null)

```

```

/// <summary>
/// Stops all currently active animation composers across every
layer,
/// blending them out over the specified time.
/// </summary>
/// <param name="blendOutTime">
/// The time over which the animations should fade out on all
layers.
/// </param>
public void InterruptAllAnimComposers(float blendOutTime)

```

```

/// <summary>
    /// Stops all currently active anim composers on a specific layer
    with a blend-out.
    /// </summary>
    /// <param name="blendOutTime">The time over which the animations
    should fade out.</param>
    /// <param name="layer">The layer on which to interrupt the
    animations.</param>
    public void InterruptActiveAnimComposersInLayer(float blendOutTime,
    int layer)

```

```

/// <summary>
    /// Sets the playback rate for all active anim composers across all
    layers.
    /// </summary>
    /// <param name="newRate">The new playback rate.</param>
    public void SetAnimComposerPlayerRate(float newRate)

```

```

/// <summary>
    /// Enables or disables the blocking of root motion.
    /// </summary>
    /// <param name="bEnable">True to block root motion, false to allow
    it.</param>
    public void SetRootMotionBlocked(bool bEnable) =>
    bIsRootMotionBlocked = bEnable;

```

- ✔ You can **modify** or — more safely — **extend** the **AnimCoordinator** class to customize or add internal logic. Each function includes **inline documentation**, making it easy to understand its purpose and behavior.

Example:

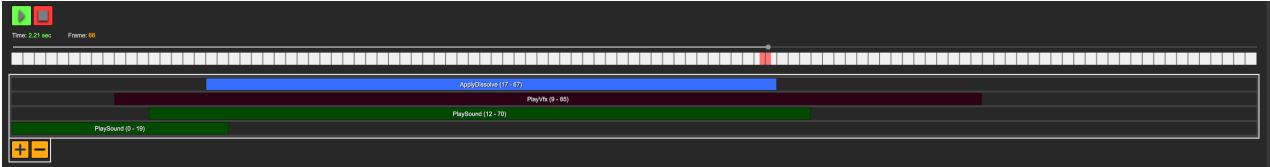
You could extend the following functions to change how root motion is applied:

- `ApplyRootMotionDisplacement` — Customize how positional movement is applied.
- `ApplyRootMotionRotation` — Adjust how rotational movement is handled.

This approach allows you to adapt the system to your game's specific needs without altering the core functionality.

ActionBlocks Overview:

Attach precise and flexible logic to animations



ActionBlocks are a powerful alternative to Unity's traditional **Animation Events**. They allow you to define custom, frame-accurate logic that runs during an animation — without relying on coroutines, delays, or additional dispatcher scripts.

Overview

An **ActionBlock** is a custom class that extends from `ActionBlock_Base`. It defines three key lifecycle methods:

- **Start** — Logic that runs when the ActionBlock begins.
- **Update** — Logic that runs every frame while the ActionBlock is active.
- **End** — Logic that runs when the ActionBlock finishes.

These ActionBlocks are assigned (via reflection) to the **AnimComposer**, where you can visually configure the exact frame range during which the ActionBlock will execute.

Why Use ActionBlocks?

Traditionally, Unity developers use **Animation Events** to trigger logic during animations. However, Animation Events have several limitations:

- They can only execute logic at a single point in time (the event's start).
- They require external scripts (e.g., Animation Behaviours or Dispatchers) to manage more complex timing.
- They lack built-in debugging or visual feedback tools.

ActionBlocks solve all of these issues by combining both **timing** and **logic** into a single, reusable unit.

Key Benefits

- ✓ **Multi-Phase Logic** — Define logic for **Start**, **Update**, and **End** phases, not just a single trigger.
 - ✓ **Visual Timeline Control** — Easily set when the logic runs directly inside the **AnimComposer** editor.
 - ✓ **Self-Contained** — No need for coroutines, timers, or extra behaviour scripts.
 - ✓ **Debug-Friendly** — Preview the effects of an ActionBlock (e.g., VFX, SFX, hitboxes, motion warping, etc.) without having to run the game.
 - ✓ **Highly Flexible** — Any gameplay or visual logic can be implemented as an ActionBlock.
-

Example Use Cases

ActionBlocks can be used to:

- Play visual or sound effects
- Spawn projectiles
- Apply procedural shader materials (e.g., fade-in effects)
- Trigger player buffs or temporary states
- Align or warp the player toward a target
- Smoothly interpolate to an interaction point
- Block or buffer input
- Provide invulnerability frames

...and much more.

Summary

In short, **ActionBlocks** replace and vastly improve upon Unity's Animation Events. They give you **frame-perfect control** over when and how gameplay logic executes within an animation, all while keeping your workflow clean, visual, and maintainable.

Personal Note:

I use ActionBlocks more than any other feature.

Any logic I want to execute during an animation lives inside an ActionBlock.

Creating a new ActionBlock.

Creating a New ActionBlock Class

1. In the **Project** window, right-click inside your **Scripts** folder (or any preferred location).
2. Select **Create → Scripting → C# Script** to generate a new script file.
3. Give your script a descriptive name — this will be the name of your new **ActionBlock** class.
4. Open the script and make sure your class inherits from **ActionBlock_Base**.

```
public class MyCustomActionBlock : ActionBlock_Base
{
    // Your custom logic here
}
```

- ✔ Before creating your own ActionBlocks, it's highly recommended to review the **ActionBlock_Base** script. It's concise, well-documented, and provides a clear understanding of how ActionBlocks are structured and function.

You should also take a look at the **sample ActionBlocks** included with the plugin — for example, **ActionBlock_ApplyDissolve.cs**. Studying these examples will help you see how different features are implemented and how to structure your own custom ActionBlocks effectively.

ActionBlock_Base provides a bunch of overrideable (virtual) functions. I will group them into two main categories.

Runtime Execution

These functions are executed during gameplay and define the actual behavior of your **ActionBlock** at runtime.

OnStart()

Called when: The ActionBlock begins execution.

This method initializes timing values and determines whether the ActionBlock can be activated by calling `CheckCanStartActionBlock()`, storing the result in the internal `IsActive` variable.

Important:

Always call `base.OnStart()` first when overriding this method to preserve the internal initialization logic shared by all ActionBlocks.

After calling `base.OnStart()`, you can check whether `IsActive` is `true` to decide if your custom logic should run.

This check is only relevant for `OnStart()`, since `OnUpdate()` and `OnExit()` will not be called at all if `IsActive` is `false`.

OnUpdate()

Called when: Every frame during the ActionBlock's execution.

Use this method to implement more complex, continuous logic that updates the character or system state each frame.

Examples include:

- Applying a fade effect to a material
- Using motion warping by evaluating an `AnimationCurve` and applying `Controller.Move()` based on elapsed time
- Updating attributes and UI elements (e.g., regenerating mana over time)

This function has **no default implementation**, so you **do not need** to call `base.OnUpdate()` when overriding it.

OnExit()

Called when: The ActionBlock finishes execution.

This method resets `IsActive` to `false`, ensuring that the system no longer calls `OnUpdate()` for this block.

Always call `base.OnExit()` when overriding this method to maintain proper cleanup behavior.

Use this method to handle any finalization logic, such as:

- Ensuring the character reaches its destination
- Destroying temporary objects to free memory
- Restoring character states or resetting parameters

CheckCanStartActionBlock()

Purpose: Determines whether the ActionBlock can start executing.

This function is evaluated at the start of `OnStart()`, and its return value is assigned to `IsActive`.

You should override this method to define the specific conditions required for your ActionBlock to begin execution — for example:

- Checking if a required **MonoBehaviour** component is present on the owner
 - Verifying that movement or input systems are enabled
 - Ensuring that all necessary references or fields are properly assigned
-

⚠ Centralizing all validation checks inside `CheckCanStartActionBlock()` is a good practice that promotes cleaner and more maintainable code. By doing so, you avoid writing redundant `if` statements throughout other functions such as `OnStart()`, `OnUpdate()`, or `OnExit()`.

However, keep in mind that `CheckCanStartActionBlock()` is **only evaluated once** — during `OnStart()`.

This means it should be reserved for conditions that **do not change during execution**.

If certain requirements may change while the ActionBlock is running (for example, if a referenced object gets destroyed mid-execution), you'll need to handle those cases directly within `OnUpdate()` or another appropriate function.

✅ It's best not to overcomplicate `CheckCanStartActionBlock()` with conditions such as *"another animation is not currently playing."* These kinds of coordination checks are better handled through a **state-driven abstraction**, as discussed earlier in the section *"Executing AnimComposers at Runtime."*

By keeping `CheckCanStartActionBlock()` focused only on static, one-time validations, your code remains simpler, cleaner, and easier to maintain.

Debug

These are functions that will only execute in editor. They are usefull for previewing the effects of an ActionBlock in the AnimComposer preview window, without the need to run the game and play the animation at runtime.

It provides access to the PreviewRenderUtility (think of it as the scene inside the preview window), and the PreviewObject, so you can use it to instance any kind of GameObject (vfx, prop, hitbox...) and modify the PreviewModel (attaching instanced elements, changing materials, shrinking or stretching, fading...)

✔ Jus like in a normal scene, when these preview objects are instantiated (calling `previewRenderUtility.InstantiatePrefabInScene`), their attached logic (Monobehaviours) will also execute. This can be usefull to visualize, not just the instanced object, but also any perceivable modification caused by its components' Awake or OnStart methods.

⚠ Just understand that in Editor Mode, Update functions will not really work in the usual way. There is not a deltaTime being passed every frame. If you want to preview any "update effect" take advantage of the deltaTime parameter in the OnDebugUpdate dunction. This will work since it will approximate the delta time through the animation length and rate and the delta between the current frame and the previous frame.

i For previewing sound, don't try to spawn a new AudioSource. It won't work. Just play sounds in the provided debugAudioSource variable.

- OnDebugStart: Called when starting the action block in debug mode. Initializes debug-specific fields and calculates duration.

Make sure you call `base.OnDebugStart` when overriding.

- OnDebugUpdate: Called every frame in debug mode. Use deltaTime to preview updates in any logic.
- OnDebugExit: Called when exiting DebugMode.

❗ It's especially important to clean up after yourself in `OnDebugExit` (e.g. Destroying instantiated prefabs), since any memory leak will be problematic. `PreviewRenderUtility` has ways of getting rid of orphaned objects after a while, but it will riddle you with warnings and the performance in-editor might be affected.

Don't stress about it, you will likely get Warnings reminding you of this. Just make sure you destroy any Instantiated object or nullify `new()` objects you have created for debug purposes in `OnDebugExit`.

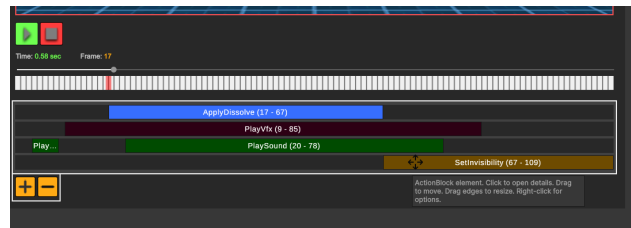
✅ You may have noticed a `Color` attribute defined at the top of custom **ActionBlock** classes.

This attribute isn't just decorative — it assigns a **unique color** to your ActionBlock, making it easy to visually identify within the **AnimComposer** tracks.

Be sure to define a distinct color for each custom ActionBlock you create. Doing so improves clarity when working with complex compositions containing multiple ActionBlocks.



Picking color in the BlockColor attribute



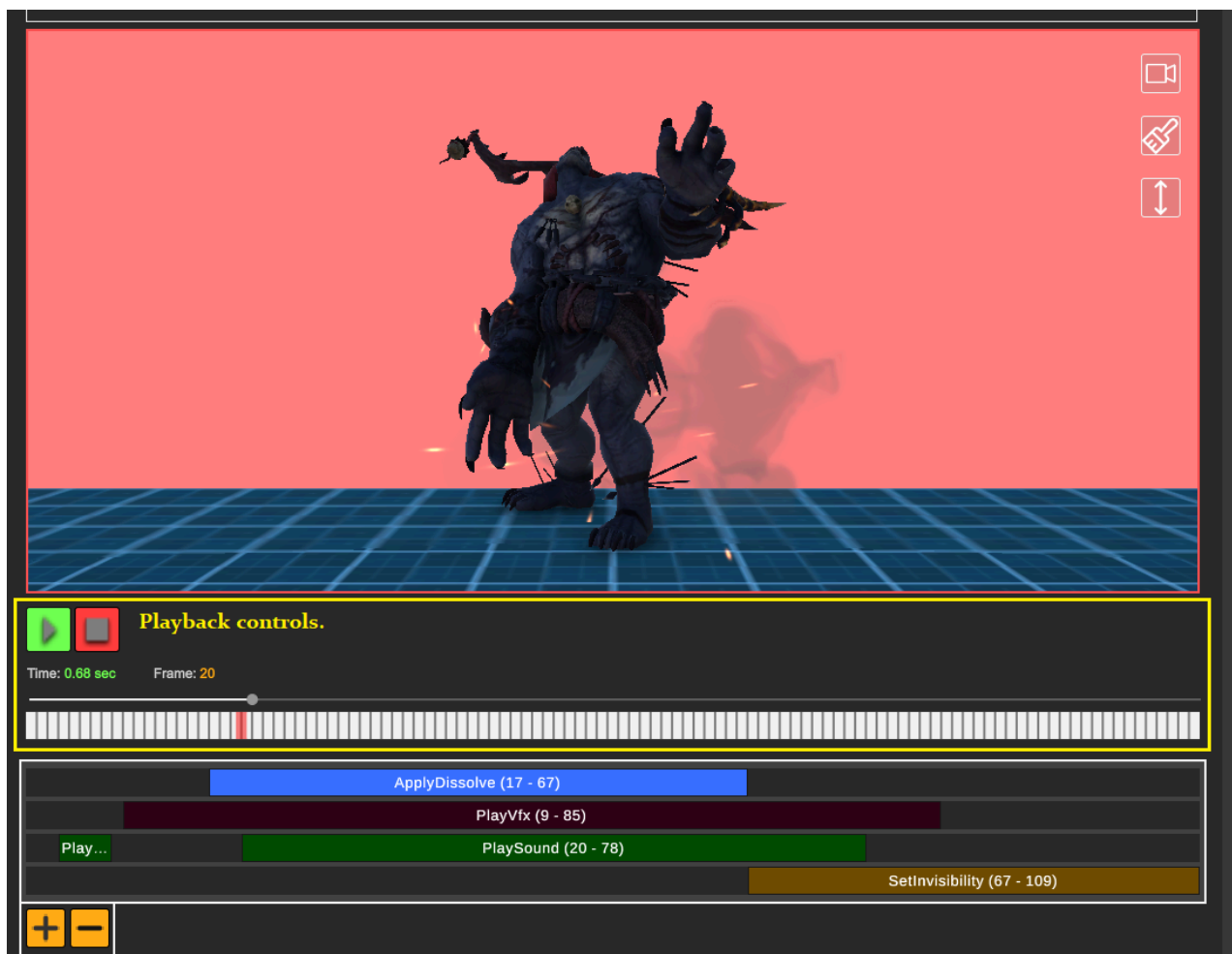
ActionBlock visualized in the AnimComposer

Assigning ActionBlocks to AnimComposer.

Now that you have created and defined functionality for your custom ActionBlock, you will see how you can link it to a particular animation through the AnimComposer.

Now that you've created and implemented functionality for your custom **ActionBlock**, the next step is to **link it to a specific animation** using the **AnimComposer**.

Before doing so, it's important to understand the **buttons and sliders** related to animation playback that are available in the **AnimComposer ScriptableObject**. These controls allow you to preview, scrub, and fine-tune your animation timing — which is essential when synchronizing your ActionBlocks with key animation moments.



Yellow box highlights playback controls

- **Play / Pause Button**

Toggles between playing and pausing the animation at the current frame.

Use this to preview how your ActionBlocks behave in real time or to pause and inspect a specific frame.

- **Stop Button**

Stops playback and resets the animation to the **first frame**.

This is useful when you want to restart playback from the beginning or make timing adjustments from the start of the clip.

- **Time and Frame Indicators**

Display the **current playback time** (in seconds) and the **current frame index**.

These indicators help you track your position within the animation precisely, especially when aligning ActionBlocks with specific keyframes.

- **Time Bar**

A continuous bar that allows **click-and-drag** control for fine-tuning playback.

Dragging the knob adjusts the animation time smoothly, and the **Frame Bar** will automatically update to match the nearest frame.

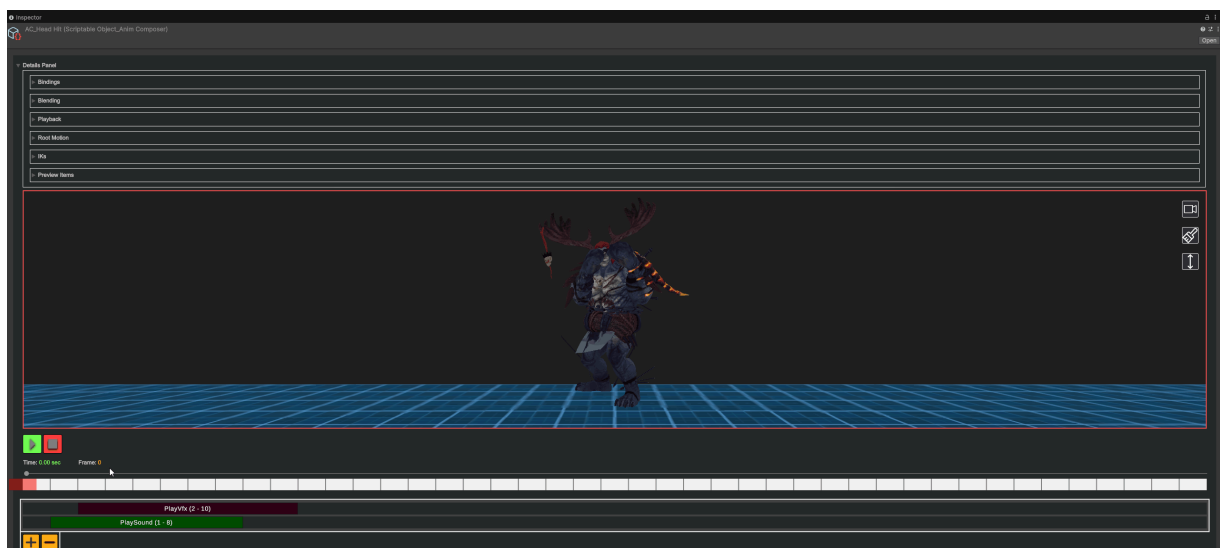
- **Frame Bar**

A discrete control that lets you jump directly to a specific **frame** rather than a time value.

Selecting a frame here will update the **Time Bar** accordingly.

Use this when you want precise, frame-level control over your animation timeline.

Would you like me to add a small diagram-style section (e.g., a labeled image reference or placeholder)



Adding Your Custom ActionBlock

Now that you know how to navigate through different moments in an animation, you can add your **custom ActionBlock** to the **AnimComposer** timeline. Follow the steps below:

1. Select the Desired Frame

Use the **Frame Bar** to choose the exact frame where you want your ActionBlock to start.

This determines when your ActionBlock will trigger during the animation.

2. Add a new Action Track

ActionBlocks are placed inside **ActionTracks**.

If you don't already have a track for your block, you'll need to create one.

To add a new **ActionTrack**:

- Click the **yellow "+" button** at the bottom of the **Tracks Container**.
 - If you already have an existing track, you can insert a new one below it by:
 - Selecting the track and pressing **+** or **I**, or
 - Right-clicking the track and choosing **"Insert New Track"** from the context menu.
-

3. Add the ActionBlock

Once your track is created:

1. Right-click on a position within the new track that roughly matches the **horizontal position** of your current frame marker.
2. Select **"Add New ActionBlock"** from the context menu.
3. Choose your **custom ActionBlock** from the list of available options.

Your **ActionBlock** will now appear in the timeline at the selected frame, ready to execute when the animation reaches that point.



For better organization, you can group your **ActionBlocks** into **subcategories** within the context menu.

This makes it easier to locate specific types of **ActionBlocks** when working with complex projects that contain many of them.

This system works similarly to the **BlockColor** attribute.

To categorize your **ActionBlock**, simply place the **ActionSubGroup** attribute above your class definition:

```
[ActionSubGroup("FX")]
public class ActionBlock_ApplyDissolve : ActionBlock_Base
{
    // Custom functionality here
}
```

In this example, all **ActionBlocks** labeled with **"FX"** will appear together under a subgroup named **"FX"** in the context menu.

Use descriptive subgroup names that represent the purpose of your **ActionBlocks** — for instance:

- **FX** → for visual effects or material-related functionality
- **Audio** → for sound or music-related actions
- **Gameplay** → for movement, combat, or logic-driven functionality



Right-clicking on **ActionTracks** or **ActionBlocks** will open a **context menu** showing all available actions for that element.

This menu not only lists the operations you can perform (such as inserting, duplicating, or deleting), but also conveniently displays the **keyboard shortcuts** for each action — shown in parentheses next to their names.

Exploring these menus is a great way to discover useful workflow shortcuts and streamline your editing process.

Adjusting and Configuring Your ActionBlock

Once your **ActionBlock** has been added to a track, you can **move** and **resize** it to cover the desired frame range.

The selected range (start and end frames) is displayed next to the ActionBlock's name label in the timeline.

Moving the ActionBlock

- Hover your cursor over the **drag icon**  that appears when you move across the ActionBlock.
- **Click and drag** to reposition it along the timeline to the desired start frame.

Resizing the ActionBlock

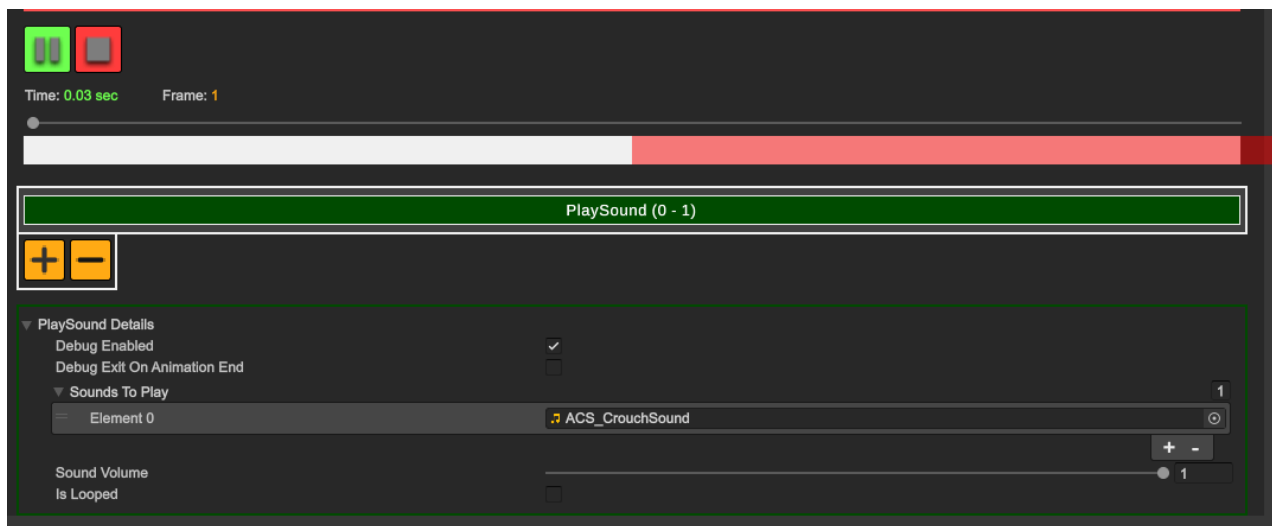
- Hover over either **side margin** of the ActionBlock until the **stretch icon**  appears.
- **Click and drag** the edge to stretch or shrink its duration, adjusting the frame range it covers.

Editing ActionBlock Properties

After selecting your ActionBlock, a **properties panel** will appear **below the ActionTracksContainer**.

This panel displays all **public fields** defined in your ActionBlock script.

Here, you can configure and fine-tune any key parameters — such as timing values, references, materials, or effect settings — to customize the ActionBlock's behavior.



Shared Variables in `ActionBlock_Base`

All **ActionBlocks** inherit two common variables from `ActionBlock_Base`.

These variables control how your ActionBlock behaves during **animation preview** inside the **AnimComposer**, particularly for testing and debugging purposes.

1. `DebugEnabled`

Determines whether the `OnDebugStart()`, `OnDebugUpdate()`, and `OnDebugExit()` functions are called as the animation updates in the **AnimComposer**.

These functions are used **exclusively for previewing** your ActionBlock's behavior in the editor (they are not called at runtime).

In the *Creating a New ActionBlock* section, you learned how to use these debug functions to visualize and fine-tune your ActionBlock's effects.

This feature is extremely useful for timing-sensitive elements — for example:

- Perfectly aligning a **sound effect** with a specific animation frame.
- Triggering a **VFX** (such as a particle burst or dissolve effect) at the exact desired moment.

Taking advantage of debug previews helps you achieve precise synchronization before running the full game.

2. `DebugExitOnAnimationEnd`

Controls when `OnDebugExit()` is called while previewing an animation.

- **If `false`:**
`OnDebugExit()` is called immediately when the ActionBlock's frame range is surpassed.
- **If `true`:**
`OnDebugExit()` is delayed until the **entire animation** finishes and loops back to the beginning.

This is particularly useful when you want to **keep visual effects visible** for as long as possible during a preview.

In runtime scenarios, you might treat such effects as **fire-and-forget**, allowing them to persist independently of the animation.

However, when previewing in the editor, it's your responsibility to ensure that:

- **Memory is properly freed**, and
- **Temporary preview instances** (such as spawned VFX or objects) are **destroyed** when the animation ends.

Using `DebugExitOnAnimationEnd` wisely allows you to extend preview visibility without causing unnecessary editor clutter or memory leaks.

Styling the AnimComposer Editor.

The **AnimComposer** custom editor gives you direct access to all its **visual elements** and **style sheets**, making it easy to modify the interface.

To begin editing, simply select the file **AnimComposerEditor (Visual Tree Asset)**.

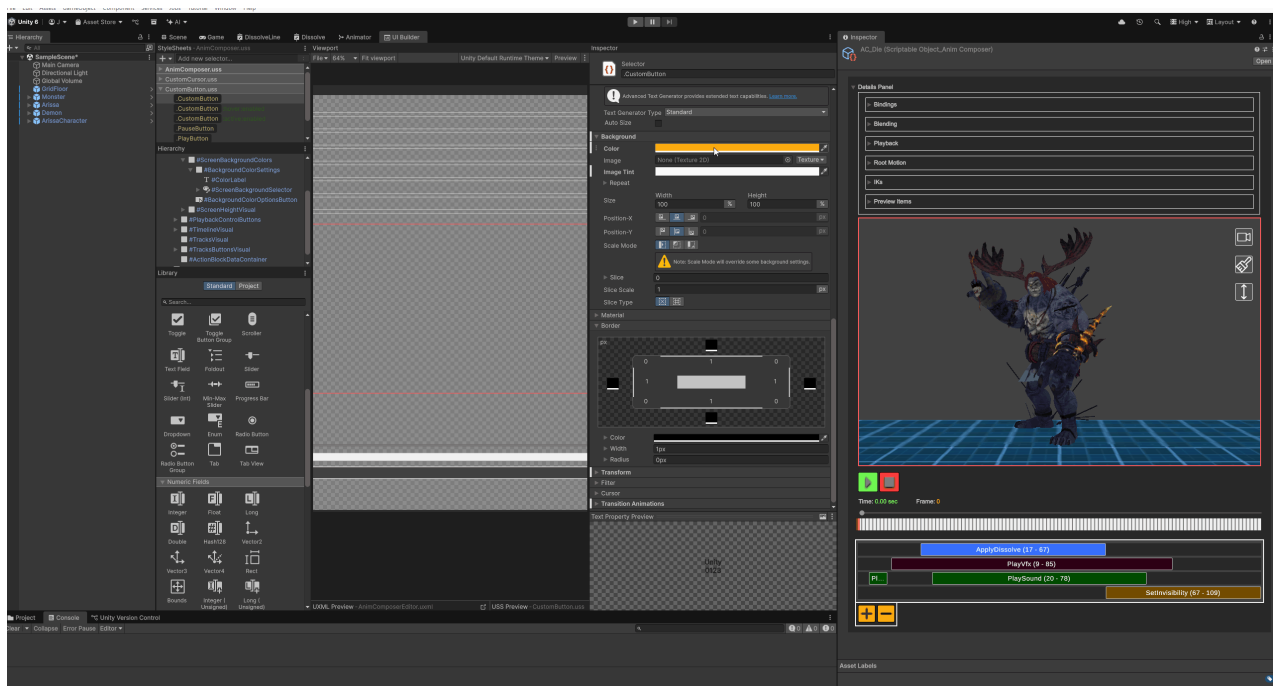
If you'd like to learn more about how Unity's **UI Toolkit** works — including how to style visual elements — check out Unity's official [UI Toolkit E-book](#).

Prefer learning through video? There's also a helpful video tutorial that explains [UI Toolkit](#) in the context of custom editors.

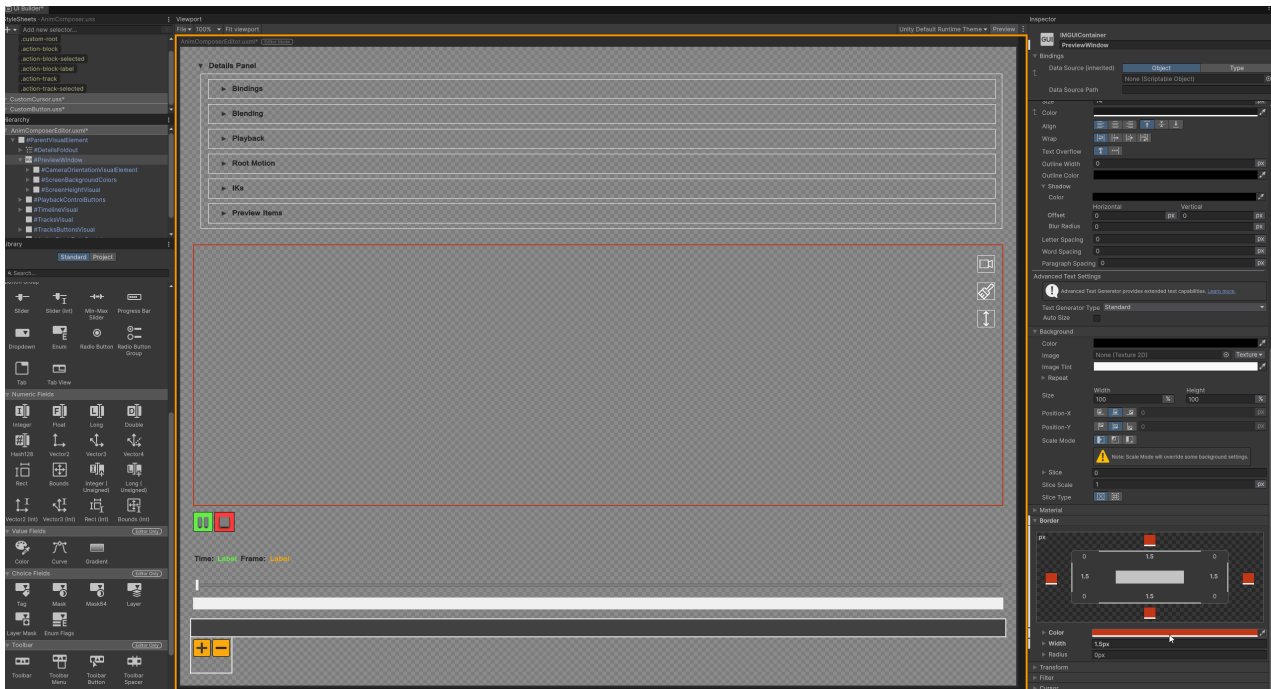
Because this is a fairly deep topic, we won't cover every detail of how the system works here.

Instead, we'll focus on a few simple visual modifications, all performed intuitively through the UI Builder Asset.

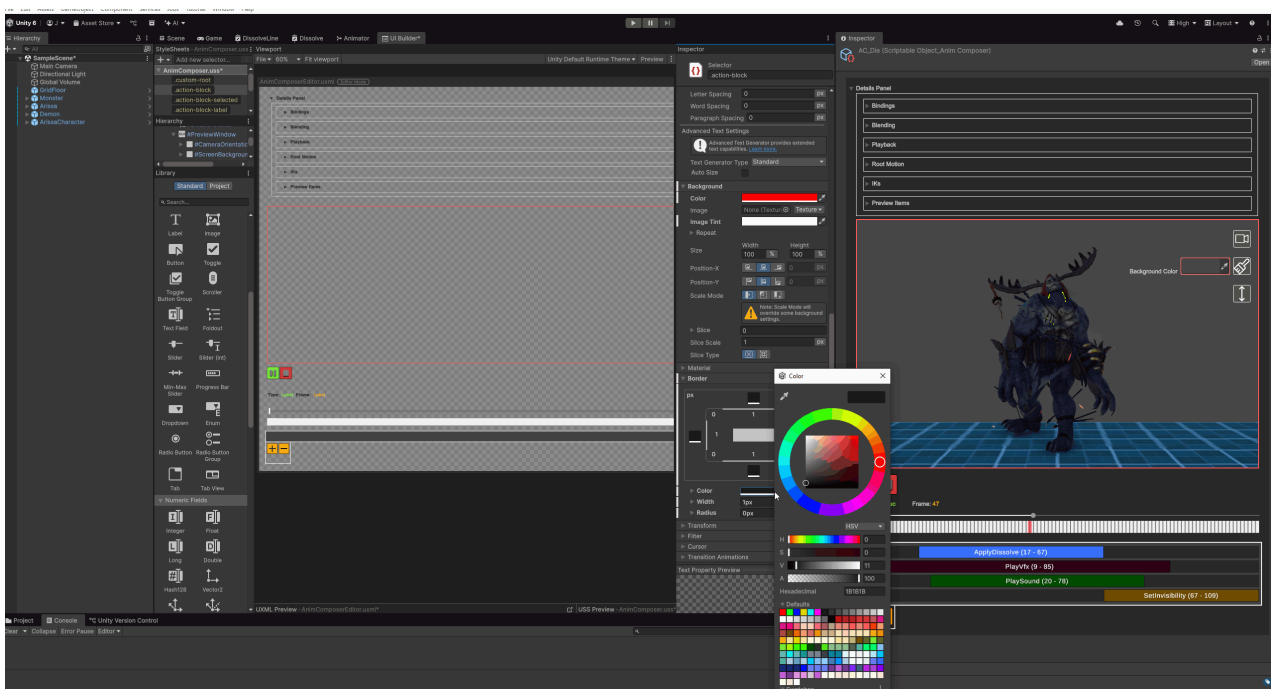
- ❗ You can also modify the **.UXML** files to adjust the **editor's layout and structure**, or the **.USS** style sheets to change the **visual appearance and aesthetics**.



Modifying the base button color by selecting and editing the **.CustomButton** style class.



Changing the Animation Preview Window border color by selecting and editing the Visual Element.



Changing the ActionBlocks border by editing the `.action-block` style class.

I'm showing only simple examples here, but the styling possibilities go far beyond color or border width. You can modify almost any aspect of the interface — even the layout and distribution of certain elements.

Keep in mind, however, that `AnimationPreviewInspector.cs` binds functionality to specific visual elements. For these to work correctly, their **assigned names must remain unchanged**.

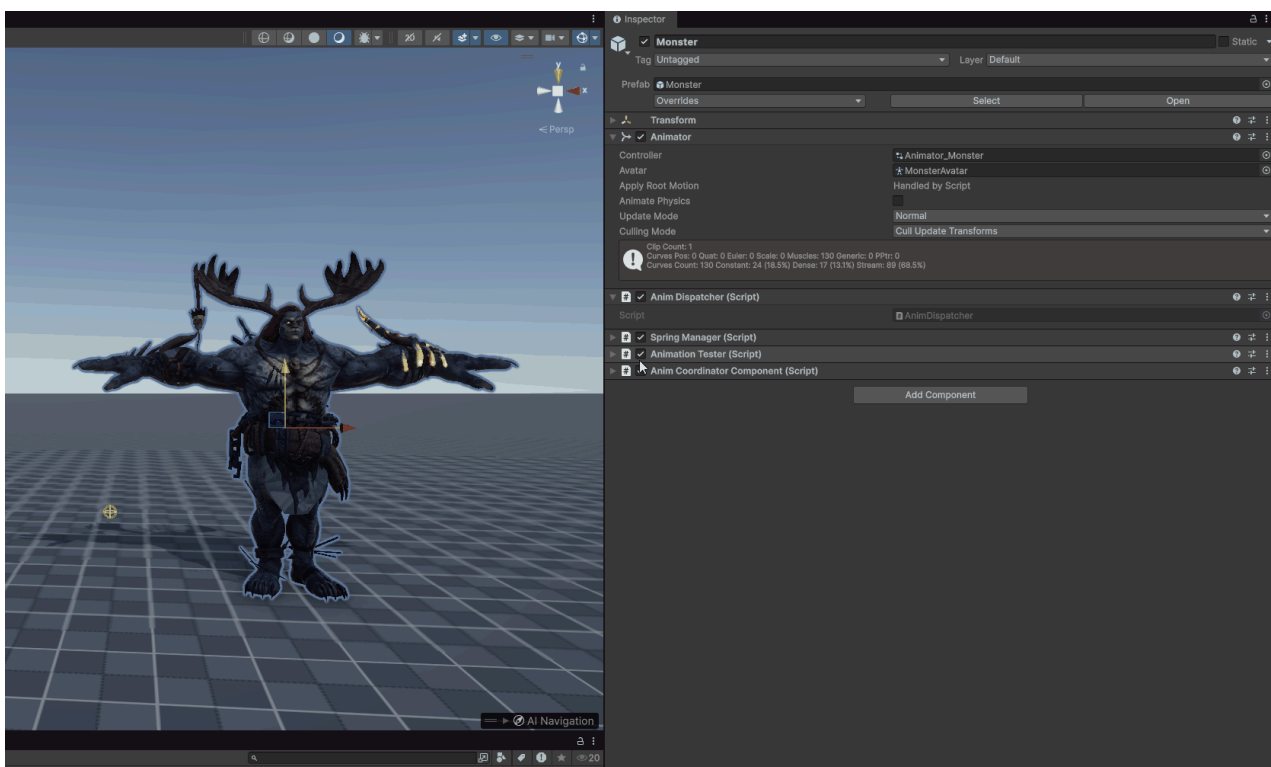
You can also modify the `AnimationPreviewInspector.cs` script itself to extend functionality, though it's strongly recommended to **back up the default version first**.

Sample Material

I recommend opening the **Sample Scene** to explore the setup. Inside, you'll find **three characters**:

- One using a **CharacterController** component
- One using a **Rigidbody** component
- And one that's completely **bare**

Each character includes a sample script I created called **AnimationTester**, which allows you to quickly test animations by binding different **AnimComposers** to different input keys.



Some of the assigned **AnimComposers** are **full animations**, while others are **additive blends** or **bone-specific overrides**. This setup lets you easily observe how different animation types mix together in real time.

All characters share the same array of **AnimComposers**. This demonstrates that a single AnimComposer can work across multiple **Humanoid** characters—regardless of the **Preview Model** selected for it.

You only need to create a **character-specific AnimComposer** when the **ActionBlock settings** should differ between characters. For example:

- A larger character may cover more distance due to **Motion Warping**.
- A particular character might trigger **unique sounds** or **custom effects** on certain actions (like dying or attacking).

Feel free to **inspect and modify** these sample characters. You can insert your own custom **AnimComposers** into the **AnimationTester** component to see how they behave, or use one of the **template characters** as a foundation for building your own custom setup.

